

Below are professionally written Java solutions to your questions.

1. Program to Convert Numerical Score to Letter Grade (A–F)

Java

Copy code

```
import java.util.Scanner;
```

```
public class GradeCalculator {  
    public static void main(String[] args) {  
        Scanner sc = new Scanner(System.in);  
        System.out.print("Enter numerical  
score: ");  
        int score = sc.nextInt();  
  
        if (score >= 90 && score <= 100)  
            System.out.println("Grade: A");  
        else if (score >= 80)  
            System.out.println("Grade: B");  
        else if (score >= 70)  
            System.out.println("Grade: C");
```

```
else if (score >= 60)
    System.out.println("Grade: D");
else if (score >= 0)
    System.out.println("Grade: F");
else
    System.out.println("Invalid score.");
}
}
```

2. Positive, Negative, or Zero

Java

Copy code

```
import java.util.Scanner;
```

```
public class NumberCheck {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter a number: ");
        double num = sc.nextDouble();

        if (num > 0)
            System.out.println("Positive");
```

```
else if (num < 0)
    System.out.println("Negative");
else
    System.out.println("Zero");
}
}
```

3. Display Initials in Uppercase

Java

Copy code

```
import java.util.Scanner;
```

```
public class Initials {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter full name: ");
        String name = sc.nextLine();

        String[] parts = name.split(" ");
        for (String part : parts) {
```

```
System.out.print(Character.toUpperCase(p
```

```
art.charAt(0)));  
    }  
}
```

```
}
```

3i. Handle Extra Spaces

Java

Copy code

String name =

```
sc.nextLine().trim().replaceAll("\\s+", " ");
```

```
String[] parts = name.split(" ");
```

3ii. Palindrome Check

Java

Copy code

```
public class PalindromeCheck {
```

```
    public static void main(String[] args) {
```

```
        Scanner sc = new Scanner(System.in);
```

```
        System.out.print("Enter word: ");
```

```
        String word = sc.nextLine();
```

```
        String reversed = new
```

```
StringBuilder(word).reverse().toString();
```

```
        if (word.equalsIgnoreCase(reversed))
            System.out.println("Palindrome");
        else
            System.out.println("Not
Palindrome");
    }
}
```

3iii. Compare Two Strings Using equals()
Java

Copy code

```
String str1 = sc.nextLine();
```

```
String str2 = sc.nextLine();
```

```
if (str1.equals(str2))
```

```
    System.out.println("Strings are equal");
```

```
else
```

```
    System.out.println("Strings are not
equal");
```

3iv. Remove Leading and Trailing Spaces
Java

Copy code

```
String sentence = sc.nextLine();  
System.out.println(sentence.trim());
```

3v. Replace Spaces with Underscores

Java

Copy code

```
String input = sc.nextLine();  
System.out.println(input.replace(" ", "_"));
```

3vi. Display Index of '@' in Email

Java

Copy code

```
String email = sc.nextLine();  
int index = email.indexOf('@');  
System.out.println("Index of @: " + index);
```

3vii. Check if Sentence Contains "java"

Java

Copy code

```
String sentence =  
sc.nextLine().toLowerCase();  
if (sentence.contains("java"))  
    System.out.println("Contains 'java'");
```

else

```
    System.out.println("Does not contain  
'java'");
```

4. File Reading with Exception Handling Java

Copy code

```
import java.io.*;
```

```
import java.util.Scanner;
```

```
public class FileReadExample {  
    public static void main(String[] args) {  
        Scanner sc = new Scanner(System.in);  
        System.out.print("Enter filename: ");  
        String fileName = sc.nextLine();  
  
        try {  
            BufferedReader br = new  
BufferedReader(new  
FileReader(fileName));  
            String line = br.readLine();  
            int number = Integer.parseInt(line);
```

```
        System.out.println("Number read: "
+ number);
        br.close();
    } catch (FileNotFoundException e) {
        System.out.println("File not found.");
    } catch (NumberFormatException e) {
        System.out.println("Invalid number
format.");
    } catch (IOException e) {
        System.out.println("IO Error
occurred.");
    }
}
}
```

5. Fixed vs Dynamic

The primary disadvantage of a standard Java array is its fixed size. Once created, its length cannot be changed. The ArrayList class solves this limitation by providing dynamic resizing, allowing elements to be added or removed at

runtime.

6. Indexing

If an array has a length of 1, the index of the last element is:

Copy code

0

(Java arrays are zero-indexed.)

7. Continuous Memory

Array elements are stored in continuous memory because Java allocates them in sequential memory locations. This allows fast access using index calculations:

$$\text{Address} = \text{BaseAddress} + (\text{Index} \times \text{ElementSize})$$

8. LIFO vs FIFO

Stack (LIFO – Last In First Out): The last element added is the first removed.

Queue (FIFO – First In First Out): The first element added is the first removed.

9. Linear Search Method

Java

Copy code

```
public int findIndex(int[] array, int target) {  
    for (int i = 0; i < array.length; i++) {  
        if (array[i] == target)  
            return i;  
    }  
    return -1;  
}
```

10. Average Calculator

Java

Copy code

```
public class AverageCalculator {  
    public static void main(String[] args) {  
        double[] numbers = {10.5, 20.0, 30.5,  
40.0, 50.0};  
        double sum = 0;  
  
        for (double num : numbers) {  
            sum += num;  
        }  
    }  
}
```

```
        double average = sum /  
numbers.length;  
        System.out.println("Average: " +  
average);  
    }  
}
```

11. ArrayList Transformation

Java

Copy code

```
import java.util.*;  
  
public class ArrayListExample {  
    public static void main(String[] args) {  
        ArrayList<Integer> list = new  
ArrayList<>(Arrays.asList(1,2,3,4,5));  
  
        for (int i = 0; i < list.size(); i++) {  
            list.set(i, list.get(i) * 2);  
        }  
    }  
}
```

```
        Iterator<Integer> it = list.iterator();
        while (it.hasNext()) {
            System.out.println(it.next());
        }
    }
}
```

12. Object Sorting Using Bubble Sort

Java

Copy code

```
class Student {
    int grade;

    Student(int grade) {
        this.grade = grade;
    }
}

public class StudentSort {
    public static void main(String[] args) {
        Student[] students = {
            new Student(75),
```

```
    new Student(90),  
    new Student(60)  
};
```

```
// Bubble Sort  
for (int i = 0; i < students.length - 1; i+  
+) {  
    for (int j = 0; j < students.length - i -  
1; j++) {  
        if (students[j].grade > students[j +  
1].grade) {  
            Student temp = students[j];  
            students[j] = students[j + 1];  
            students[j + 1] = temp;  
        }  
    }  
}
```

```
for (Student s : students) {  
    System.out.println(s.grade);  
}
```

}

}